

Lab 2.2 refactorización

Luis Pedro Hernández Maldonado

Luis Pedro Figueroa Monroy

(No encontramos como subir un comentario solo con la dirección de github)

https://github.com/LuisPHernandez/MarketEHU.git

Refactorización Luis Pedro Hernández

Write short units of code

1. El método resolveErreklamazioa() ocupa 52 líneas de código:

```
public boolean resolveErreklamazioa(Integer saleNumber, boolean accept) {
    try {
        db.getTransaction().begin();
        Sale sale = db.find(Sale.class, saleNumber);
        if (sale == null || sale.getErreklamazioa() == null) {
            db.getTransaction().rollback();
            return false;
        }

        if (accept) {
            Seller buyer = sale.getBuyer();
            Seller seller = sale.getSeller();

            if (buyer != null && seller != null) {
                buyer.addMoney(sale.getPrice());
                seller.addMoney(-sale.getPrice());

                Mugimenduak mBuyer = new Mugimenduak("ITZULKETA", new java.util.Date(), buyer);
                mBuyer.setSale(sale);
                db.persist(mBuyer);

                Mugimenduak mSeller = new Mugimenduak("ITZULKETA_KENTZEA", new java.util.Date(), seller);
                mSeller.setSale(sale);
                db.persist(mSeller);

                buyer.getPurchasedSales().remove(sale);
                sale.setBuyer(null);

                db.merge(buyer);
                db.merge(seller);
            }
        }

        Erreklamazioa err = sale.getErreklamazioa();
        sale.setErreklamazioa(null);
        db.merge(sale);

        if (err != null) {
            Erreklamazioa dbErr = db.find(Erreklamazioa.class, err.getId());
            if (dbErr != null)
                err.setTratatuta(true);
            db.merge(err);
        }

        db.getTransaction().commit();
        return true;
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

2. El código refactorizado:

```java
public boolean resolveErreklamazioa(Integer saleNumber, boolean accept) {
    try {
        db.getTransaction().begin();
        Sale sale = db.find(Sale.class, saleNumber);
        if (sale == null || sale.getErreklamazioa() == null) {
            db.getTransaction().rollback();
            return false;
        }
        applyResolution(sale, accept);
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        return rollbackAndFail(e);
    }
}

private void applyResolution(Sale sale, boolean accept) {
    if (accept) {
        refundSale(sale);
    }
    closeErreklamazioa(sale);
}

private void refundSale(Sale sale) {
    Seller buyer = sale.getBuyer();
    Seller seller = sale.getSeller();
    if (buyer == null || seller == null) {
        return;
    }
    buyer.addMoney(sale.getPrice());
    seller.addMoney(-sale.getPrice());
    persistMugimendu("ITZULKETA", buyer, sale);
    persistMugimendu("ITZULKETA_KENTZEA", seller, sale);
    buyer.getPurchasedSales().remove(sale);
    sale.setBuyer(null);
    db.merge(buyer);
    db.merge(seller);
}
```

```java
private void closeErreklamazioa(Sale sale) {
    Erreklamazioa err = sale.getErreklamazioa();
    sale.setErreklamazioa(null);
    db.merge(sale);

    if (err != null) {
        Erreklamazioa dbErr = db.find(Erreklamazioa.class, err.getId());
        if (dbErr != null)
            err.setTratatuta(true);
        db.merge(err);
    }
}

private boolean rollbackAndFail(Exception e) {
    e.printStackTrace();
    db.getTransaction().rollback();
    return false;
}
```

3. **Code smell:** función demasiado larga. resolveErreklamazioa tenía 52 líneas (límite: 15) y mezclaba tres responsabilidades: validar la reclamación, devolver el dinero al comprador y cerrar la reclamación.
   Refactorización: Extract Method aplicado varias veces. refundSale contiene la devolución (saldos, movimientos y desvincular al comprador), closeErreklamazioa el cierre de la reclamación, applyResolution decide si hay devolución, y rollbackAndFail el tratamiento de errores del catch, que además es reutilizable por el resto de métodos. El método pasa de 52 a 15 líneas y ninguna de las unidades nuevas supera las 15. El comportamiento no cambia.
4. **Miembro:** Luis Pedro Hernández

Duplicate code

1. El mismo bloque de 3 líneas aparece 5 veces: en buyProduct, buyBasket, confirmArrival y dos veces en resolveErreklamazioa:

```
Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new java.util.Date(), buyer);
mugimendu.setSale(sale);
db.persist(mugimendu);
```

2. El código refactorizado es un método simple que ejecuta estas 3 líneas:

```
private void persistMugimendu(String mota, Seller seller, Sale sale) {
    Mugimenduak mugimendu = new Mugimenduak(mota, new java.util.Date(), seller);
    mugimendu.setSale(sale);
    db.persist(mugimendu);
```

3. **Code smell:** código duplicado. La creación y persistencia de un movimiento (crear el Mugimenduak, asociarle la venta y hacer persist) estaba copiada en cinco lugares de DataAccess (buyProduct, buyBasket, confirmArrival y dos veces en resolveErreklamazioa).
   Refactorización: Extract Method. Se crea el método privado persistMugimendu(String mota, Seller seller, Sale sale) y las cinco copias se sustituyen por una llamada que solo indica el tipo de movimiento, el usuario y la venta. El comportamiento no cambia.
4. **Miembro:** Luis Pedro Hernández

Write simple units of code

1. El método addToBasket tenía más ramas de las permitidas:

```
public boolean addToBasket(String buyerEmail, Integer saleNumber) {
    try {
        db.getTransaction().begin();
        Seller buyer = db.find(Seller.class, buyerEmail);
        Sale sale = db.find(Sale.class, saleNumber);

        if (buyer == null || sale == null || sale.getBuyer() != null) {
            db.getTransaction().rollback();
            return false;
        }

        List<Sale> basket = buyer.getBasket();

        for (Sale s : basket) {
            if (s.getSaleNumber().equals(saleNumber)) {
                db.getTransaction().rollback();
                return false;
            }
        }

        if (basket.isEmpty()) {
            basket.add(sale);
            db.merge(buyer);
            db.getTransaction().commit();
            return true;
        }

        Seller firstSeller = buyer.getBasket().get(0).getSeller();

        if (!sale.getSeller().equals(firstSeller)) {
            return false;
        }

        buyer.addToBasket(sale);
        db.merge(buyer);

        db.getTransaction().commit();
        return true;

    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

2. El código refactorizado:

```java
public boolean addToBasket(String buyerEmail, Integer saleNumber) {
    try {
        db.getTransaction().begin();
        Seller buyer = db.find(Seller.class, buyerEmail);
        Sale sale = db.find(Sale.class, saleNumber);

        if (!canBeAddedToBasket(buyer, sale, saleNumber)) {
            db.getTransaction().rollback();
            return false;
        }

        List<Sale> basket = buyer.getBasket();

        if (basket.isEmpty()) {
            basket.add(sale);
            db.merge(buyer);
            db.getTransaction().commit();
            return true;
        }

        Seller firstSeller = basket.get(0).getSeller();

        if (!sale.getSeller().equals(firstSeller)) {
            return false;
        }

        buyer.addToBasket(sale);
        db.merge(buyer);

        db.getTransaction().commit();
        return true;

    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

```java
private boolean canBeAddedToBasket(Seller buyer, Sale sale, Integer saleNumber) {
    return buyer != null && sale != null && sale.getBuyer() == null
            && !isInBasket(buyer.getBasket(), saleNumber);
}

private boolean isInBasket(List<Sale> basket, Integer saleNumber) {
    for (Sale s : basket) {
        if (s.getSaleNumber().equals(saleNumber)) {
            return true;
        }
    }
    return false;
}
```

3. **Code smell:** función compleja. addToBasket tenía 8 puntos de decisión (límite: 4): un if con dos ||, un for con un if dentro, otros dos if y el catch.
   Refactorización: Extract Method. La comprobación de si la venta puede añadirse a la cesta se extrae a canBeAddedToBasket (usuario y venta existen, la venta no está vendida y no está ya en la cesta), y la búsqueda en la cesta a isInBasket. addToBasket queda con 4 puntos de decisión, canBeAddedToBasket con 3 e isInBasket con 2. Se mantiene el orden de las comprobaciones, así que el comportamiento no cambia.
4. **Miembro:** Luis Pedro Hernández

Keep unit interfaces small

1. CreateSale() toma 7 parámetros:

```
public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail,
        File file) throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {
```

2. El código refactorizado:

```
public class SaleData {
    private final String title;
    private final String description;
    private final int status;
    private final float price;
    private final Date pubDate;

    public SaleData(String title, String description, int status, float price, Date pubDate) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
        this.pubDate = pubDate;
    }

    public String getTitle() { return title; }
    public String getDescription() { return description; }
    public int getStatus() { return status; }
    public float getPrice() { return price; }
    public Date getPubDate() { return pubDate; }
}
```

```
public Sale createSale(SaleData data, String sellerEmail, File file)
        throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {
```

```
public Sale createSale(String title, String description,int status, float price, Date pubDate, String sellerEmail, File file)
        throws  FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {
    dbManager.open();
    Sale product=dbManager.createSale(new SaleData(title, description, status, price, pubDate), sellerEmail, file);
    dbManager.close();
    return product;
};
```

3. **Code smell:** interfaz demasiado grande. DataAccess.createSale recibía 7 parámetros (límite: 4), cinco de los cuales (título, descripción, estado, precio y fecha) describen juntos una misma cosa, los datos de la venta.
   Refactorización: Introduce Parameter Object. Se crea la clase domain.SaleData, que agrupa esos cinco datos, y createSale pasa a recibir (SaleData data, String sellerEmail, File file): 3 parámetros. Se actualiza su única llamada, en BLFacadeImplementation. La interfaz BLFacade no cambia, así que la GUI no se ve afectada. El comportamiento no cambia.
4. **Miembro:** Luis Pedro Hernández

Refactorización Luis Pedro Figueroa

Duplicate code

Los métodos getSales y getPublishedSales repiten el mismo println y el setparameter(1, "%"+desc+"%"). El bucle copía los resultados a otra lista.

```
9
0     public List<Sale> getSales(String desc) {
1         System.out.println(">> DataAccess: getProducts=> from= "+desc);
2         List<Sale> res = new ArrayList<Sale>();
3         TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE
4         query.setParameter(1, "%"+desc+"%");
5
6         List<Sale> sales = query.getResultList();
7         for (Sale sale:sales){
8             res.add(sale);
9         }
0         return res;
1     }
2
3     public List<Sale> getPublishedSales(String desc, Date pubDate) {
4         System.out.println(">> DataAccess: getProducts=> from= "+desc);
5         List<Sale> res = new ArrayList<Sale>();
6         TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE
7         query.setParameter(1, "%"+desc+"%");
8         query.setParameter(2,pubDate);
9
0         List<Sale> sales = query.getResultList();
1         for (Sale sale:sales){
2             res.add(sale);
3         }
4         return res;
5     }
```

Código refactorizado

```
public List<Sale> getSales(String desc) {
    TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE
    return searchSalesByTitle(query, desc);
}
```

```
    public List<Sale> getPublishedSales(String desc, Date pubDate) {
        TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE
        query.setParameter(2,pubDate);
        return searchSalesByTitle(query, desc);
    }
```

Método Extraido para simplificar

```java
private List<Sale> searchSalesByTitle(TypedQuery<Sale> query, String desc) {
    System.out.println(">> DataAccess: getProducts=> from= "+desc);
    List<Sale> res = new ArrayList<Sale>();
    query.setParameter(1, "%"+desc+"%");

    List<Sale> sales = query.getResultList();
    for (Sale sale:sales){
        res.add(sale);
    }
    return res;
}
```

5. **Code smell:** en los métodos getSales y getPublishedSales había 8 líneas de código repetidas. El mensaje en consola, la creación de la lista de resultados, el query, la ejecución de la consulta y la copía de los resultados en una lista. Si se quisiera cambiar esta lógica, se tendría que cambiar en 2 lados.
6. **Refactorización:** Usando Extract Method, se extrajo el bloque de código repetido a un método privado llamado SearchSalesByTitle(TypedQuery<Sale> query, String desc). Ahora, cada método solo crea su consulta y llama a searchSalesByTitle.
7. **Miembro:** Luis Pedro Figueroa

Short Units

El método acceptEskaintza tenía 40 líneas de código, pasandose del límite de 15.

```java
public boolean acceptEskaintza(Integer eskaeraId, Integer eskaintzaId) {
    try {
        // Todas las modificaciones se hacen en una única transacción:
        // o se aplican todas, o ninguna (rollback).
        db.getTransaction().begin();

        // 1. Recuperar la solicitud y la oferta de la base de datos
        Eskaera eskaera = db.find(Eskaera.class, eskaeraId);
        Eskaintza eskaintza = db.find(Eskaintza.class, eskaintzaId);

        // 2. Validaciones: ambas deben existir y la solicitud debe seguir abierta
        if (eskaera == null || eskaintza == null || eskaera.isClosed()) {
            db.getTransaction().rollback();
            return false;
        }

        // El comprador es quien creó la solicitud; el vendedor, quien hizo la oferta
        Seller buyer = eskaera.getBuyer();
        Seller seller = eskaintza.getSeller();
        float price = eskaintza.getPrice();

        // 3. Comprobar que el comprador tiene saldo suficiente para pagar la oferta
        if (buyer.getMoney() < price) {
            db.getTransaction().rollback();
            return false;
        }

        // 4. Cobro: se descuenta el precio del saldo del comprador
        buyer.addMoney(-price);

        // 5. Cerrar la solicitud para que no se acepten más ofertas
        eskaera.setClosed(true);

        // 6. Crear la venta en el vendedor a partir de los datos de la solicitud y la oferta
        //    (cantidad 1, fecha actual, sin imagen)
        String tituloSale = "[Eskaera] " + eskaera.getTitle();
        String descSale = eskaintza.getMessage();

        Sale transaccion = seller.addSale(tituloSale, descSale, 1, price, new java.util.Date(), null);
        // Asociar la venta al comprador y añadirla a su lista de compras
        transaccion.setBuyer(buyer);
        buyer.addPurchasedSale(transaccion);

        // 7. Crear el envío asociado a la venta
        Bidalketa bidalketa = new Bidalketa(transaccion);
        transaccion.setBidalketa(bidalketa);

        db.persist(bidalketa);
        db.persist(transaccion);

        // 8. Registrar el movimiento de pago del comprador, vinculado a la solicitud y la oferta
        Mugimenduak mugimendu = new Mugimenduak("ESKAINTZA_ORDAINKETA", new java.util.Date(), buyer);
        mugimendu.setEskaera(eskaera);
        mugimendu.setEskaintza(eskaintza);
        db.persist(mugimendu);

        // 9. Guardar los cambios de las entidades modificadas y confirmar la transacción
        db.merge(buyer);
        db.merge(seller);
        db.merge(eskaera);

        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        // Ante cualquier error se deshacen todos los cambios
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

Código refactorizado:

```java
public boolean acceptEskaintza(Integer eskaeraId, Integer eskaintzaId) {
    try {
        db.getTransaction().begin();
        Eskaera eskaera = db.find(Eskaera.class, eskaeraId);
        Eskaintza eskaintza = db.find(Eskaintza.class, eskaintzaId);

        if (!canAcceptEskaintza(eskaera, eskaintza)) {
            db.getTransaction().rollback();
            return false;
        }

        formalizeEskaintza(eskaera, eskaintza);
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        // Ante cualquier error se deshacen todos los cambios
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

```java
    /**
     * Registra el movimiento (Mugimenduak) de pago del comprador, de tipo

     */
    private void registerEskaintzaPayment(Seller buyer, Eskaera eskaera, Eskaintza eskaintza) {
        Mugimenduak mugimendu = new Mugimenduak("ESKAINTZA_ORDAINKETA", new java.util.Date(), buyer);
        mugimendu.setEskaera(eskaera);
        mugimendu.setEskaintza(eskaintza);
        db.persist(mugimendu);
    }
```

```java
    /**
     * Comprueba si una oferta se puede aceptar.

     */
    private boolean canAcceptEskaintza(Eskaera eskaera, Eskaintza eskaintza) {
        return eskaera != null && eskaintza != null && !eskaera.isClosed()
                && eskaera.getBuyer().getMoney() >= eskaintza.getPrice();
    }
```

```java
    /**
     * Formaliza la compra de una oferta ya validada. Debe llamarse dentro de una
     * transacción abierta.
     */
    private void formalizeEskaintza(Eskaera eskaera, Eskaintza eskaintza) {
        // El comprador es quien creó la solicitud; el vendedor, quien hizo la oferta
        Seller buyer = eskaera.getBuyer();
        Seller seller = eskaintza.getSeller();

        buyer.addMoney(-eskaintza.getPrice());
        eskaera.setClosed(true);

        createEskaeraSale(eskaera, eskaintza, seller, buyer);
        registerEskaintzaPayment(buyer, eskaera, eskaintza);

        db.merge(buyer);
        db.merge(seller);
        db.merge(eskaera);
    }
```

```java
    /**
     * Crea en el vendedor la venta correspondiente a la oferta aceptada, la
     * asigna al comprador y crea el envío (Bidalketa) asociado.
     */
    private void createEskaeraSale(Eskaera eskaera, Eskaintza eskaintza, Seller seller, Seller buyer) {
        String tituloSale = "[Eskaera] " + eskaera.getTitle();
        String descSale = eskaintza.getMessage();

        Sale transaccion = seller.addSale(tituloSale, descSale, 1, eskaintza.getPrice(), new java.util.Date(), null);
        transaccion.setBuyer(buyer);
        buyer.addPurchasedSale(transaccion);

        Bidalketa bidalketa = new Bidalketa(transaccion);
        transaccion.setBidalketa(bidalketa);

        db.persist(bidalketa);
        db.persist(transaccion);
    }
```

Code Smell: método demasiado largo. Supera el límite de 15 líneas al tener unas 40, ya que solo en este método se validaban la solicitud, la oferta y el saldo. Se hacía el cobro a comprador y se cerraba la solicitud. También, se creaba la venta y su envío, se registraba el movimiento de pago y se guardaba las entidades.
Refactorización: usando Extract method, se divide el método en cuatro:

CanAcceptEskaintza: la solicitud y la oferta existen, la solicitud está abierta y el saldo de comprador es suficiente.

CreateEskaeraSale: crea la venta en el vendedor y la asigna al comprador, al igual que su envío.

RegisterEskaintzaPayment: registra el movimiento ESKAINTZA_ORDAINKETA.

FormalizeEskainta: hacer el cobro, cierra la solicitud, llama a los 2 métodos anteriores y hace los merge necesarios.

De esta forma, acceptEskaintza queda con 15 líneas, solo gestiona la transacción y todo los demás es delegado

Miembro: Luis Pedro Figueroa

Simple Units

El método resolverReport tiene 6 condicionales superando el límite de 4.

```java
public boolean resolveReport(Integer saleNumber, Salaketa salaketa,boolean aceptar) {
    try {
        db.getTransaction().begin();
        Sale sale = db.find(Sale.class, saleNumber);

        if (sale == null || salaketa == null) {
            db.getTransaction().rollback();
            return false;
        }

        sale.getSalaketak().removeIf(
                s -> Objects.equals(s.getId(), salaketa.getId())
            );

        Salaketa s = db.find(Salaketa.class, salaketa.getId());
        if (s != null) {
            s.setTratatuta(true);
            db.merge(s);
        }

        if (aceptar) {
            Seller owner = sale.getSeller();
            if (owner != null) {
                owner.removeSale(sale);
                db.merge(owner);
            }
            db.remove(sale);
        } else {
            db.merge(sale);
        }

        db.getTransaction().commit();
        return true;

    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

Código refactorizado

```
    public boolean resolveReport(Integer saleNumber, Salaketa salaketa,boolean aceptar) {
        try {
            db.getTransaction().begin();
            Sale sale = db.find(Sale.class, saleNumber);

            if (sale == null || salaketa == null) {
                db.getTransaction().rollback();
                return false;
            }

            markSaleAsTreated(sale, salaketa);
            applyReportDecision(sale, aceptar);

            db.getTransaction().commit();
            return true;

        } catch (Exception e) {
            e.printStackTrace();
            db.getTransaction().rollback();
            return false;
        }
    }
```

Métodos extraidos

```
405         }
406
407         private void applyReportDecision(Sale sale, boolean aceptar) {
408             if (aceptar) {
409                 removeSaleFromOwner(sale);
410                 db.remove(sale);
411             } else {
412                 db.merge(sale);
413             }
414         }
```

```
    private void markSaleAsTreated(Sale sale, Salaketa salaketa) {
        sale.getSalaketak().removeIf(
                s -> Objects.equals(s.getId(), salaketa.getId())
            );

        Salaketa s = db.find(Salaketa.class, salaketa.getId());
        if (s != null) {
            s.setTratatuta(true);
            db.merge(s);
        }
    }
```

```
415
416     private void removeSaleFromOwner(Sale sale) {
417         Seller owner = sale.getSeller();
418         if (owner != null) {
419             owner.removeSale(sale);
420             db.merge(owner);
421         }
422     }
```

Code smell: función compleja. En resolveReport, habían 6 puntos de decisión superando el límite de 4. Un if con un or, el if (s != null), el if (aceptar), el if (owner !=null) anidado dentro y el catch.

Refactorizacion: usando extract method, se extraen 3 métodos:

markSaleAsTreated: quita denuncia de la venta y la marca como tratada en la base de datos.

ApplyReportDecision: si la denuncia es aceptada, se elimina la venta. Si no, solo se guarda.

RemoveSaleFromOwner: quita la venta de la lista del vendedor.

Con esta refactorización, el método resolveReport queda solamente con 3 puntos de decisión.

Miembro: Luis Pedro Figueroa

Keep unit interfaces small

Es una unidad distinta de DataAccess.createSale, refactorizada por Luis Pedro Hernández; este método pertenece a la clase auxiliar de los tests

```
63      // Crea una venta del Seller ownerEmail (siempre debe existir en la BD).
64      // Si saleNumber no es null, se fija ese número, si es null, lo genera la BD.
65      public Sale createSale(String ownerEmail, String title, String description, int status, float price,
66              Date pubDate, Integer saleNumber) {
67          db.getTransaction().begin();
68          Seller owner = db.find(Seller.class, ownerEmail);
69          Sale sale = owner.addSale(title, description, status, price, pubDate, null);
70          if (saleNumber != null) {
71              sale.setSaleNumber(saleNumber);
72          }
73          db.persist(sale);
74          db.getTransaction().commit();
75          return sale;
76      }
```

Método refactorizado

```java
    // Si saleNumber no es null, se usa ese número, si es null, lo genera la BD.
    public Sale createSale(String ownerEmail, SaleTestData data, Integer saleNumber) {
        db.getTransaction().begin();
        Seller owner = db.find(Seller.class, ownerEmail);
        Sale sale = owner.addSale(data.getTitle(), data.getDescription(), data.getStatus(), data.getPrice(), data.getPubDate(), null);
        if (saleNumber != null) {
            sale.setSaleNumber(saleNumber);
        }
        db.persist(sale);
        db.getTransaction().commit();
        return sale;
    }
```

```java
package testOperations;

import java.util.Date;

public class SaleTestData {
    private String title;
    private String description;
    private int status;
    private float price;
    private Date pubDate;

    public SaleTestData(String title, String description, int status, float price, Date pubDate) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
        this.pubDate = pubDate;
    }

    public String getTitle() {
        return title;
    }

    public String getDescription() {
        return description;
    }

    public int getStatus() {
        return status;
    }

    public float getPrice() {
        return price;
    }

    public Date getPubDate() {
        return pubDate;
    }
}
```

Code Smell: interfaz demasiado grande. El método que se usaba en los tests para crear ventas en la base de datos recibía 7 parámetros superando el límite de 4. Tantos parámetros podrían causar confusión al no saber que es cada número y confundir el orden de los parámetros.

Refactorización: usando Introduce Parameter Object, los 7 datos que describen la venta (ownerEmail, title, description, status, price, pubDate y saleNumber) se agrupan en la nueva clase testOperations.SaleTestData, haciendo que la firma pase a ser createSale(String ownerEmail, SaleTestData data, Integer saleNumber) con solo 3 parámetros. El dueño y el número de venta se dejan aparte porque no son datos de la venta en sí.

Miembro: Luis Pedro Figueroa